颜色识别

颜色识别
- 例程实验效果简介
- 代码讲解
  - 完整代码
  - 代码结构
- 添加自定义颜色

例程实验效果简介

本节例程代码位于：【源码汇总 / 05.Color / 01.color_recognition.py】

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

打开例程代码，找到THRESHOLDS数组，在数组中找到你想追踪/识别的颜色的下标

添加方法我们可以看 添加自定义颜色 部分

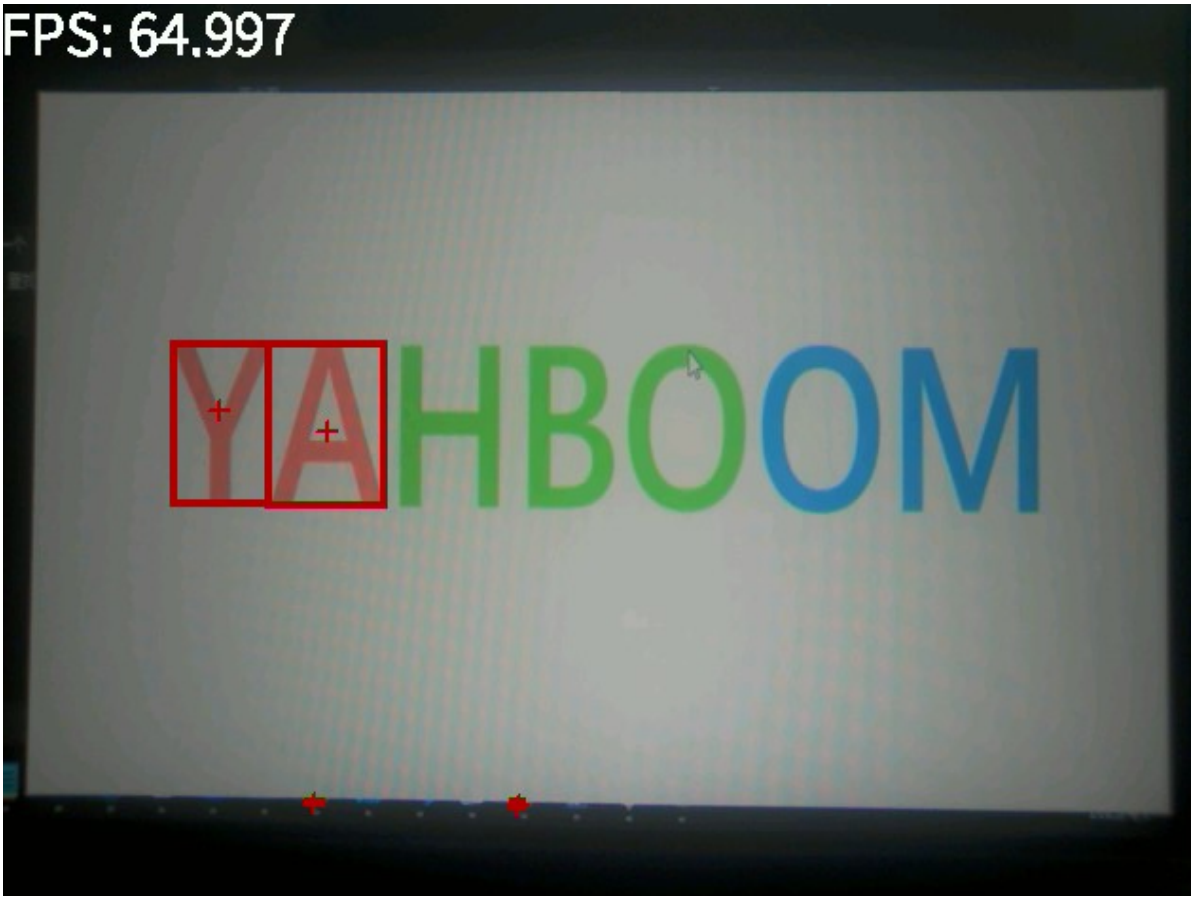
点击CanMV IDE左下角的运行按钮，

将K230的屏幕里会框出来摄像头捕获到的图像中可识别的颜色（例程默认检测红色）

> CanMV集成了基于LAB颜色模型的RGB565颜色块识别find_blobs函数

YAHBOOM

运行效果如图



代码讲解

本节我们要用的的外设主要是摄像头模块

线段检测由 K230中的 find_blobs() 方法实现，该方法属于image模块

完整代码

```
import time, os, sys
from media.sensor import *
from media.display import *
from media.media import *

# 显示参数 / Display parameters
DISPLAY_WIDTH = 640   # LCD显示宽度 / LCD display width
DISPLAY_HEIGHT = 480  # LCD显示高度 / LCD display height

# LAB颜色空间阈值 / LAB color space thresholds
# (L Min, L Max, A Min, A Max, B Min, B Max)
THRESHOLDS = [
    (0, 66, 7, 127, 3, 127),    # 红色阈值 / Red threshold
    (42, 100, -128, -17, 6, 66),     # 绿色阈值 / Green threshold
    (43, 99, -43, -4, -56, -7),       # 蓝色阈值 / Blue threshold
    (37, 100, -128, 127, -128, -27)    # 亚博智能Logo的颜色 color of YAHBOOM
]

def get_closest_rgb(lab_threshold):
    """根据LAB阈值计算最接近的RGB颜色 / Calculate closest RGB color based on LAB
threshold"""
    # 获取LAB空间的中心点值
```

```
    l_center = (lab_threshold[0] + lab_threshold[1]) // 2
    a_center = (lab_threshold[2] + lab_threshold[3]) // 2
    b_center = (lab_threshold[4] + lab_threshold[5]) // 2
    return image.lab_to_rgb((l_center,a_center,b_center))

def init_sensor():
    """初始化摄像头 / Initialize camera sensor"""
    sensor = Sensor()
    sensor.reset()
    sensor.set_framesize(width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT)
    sensor.set_pixformat(Sensor.RGB565)
    return sensor

def init_display():
    """初始化显示 / Initialize display"""
    Display.init(Display.ST7701, to_ide=True)
    MediaManager.init()

def process_blobs(img, blobs, color):
    """处理检测到的色块 / Process detected color blobs"""
    for blob in blobs:
        img.draw_rectangle(blob[0:4], color=color, thickness=4)
        img.draw_cross(blob[5], blob[6], color=color, thickness=2)

def draw_fps(img, fps):
    """绘制FPS信息 / Draw FPS information"""
    img.draw_string_advanced(0, 0, 30, f'FPS: {fps:.3f}', color=(255, 255, 255))

def main():
    try:
        # 初始化设备 / Initialize devices
        sensor = init_sensor()
        init_display()
        sensor.run()

        clock = time.clock()

        # 选择要检测的颜色索引 (0:红, 1:绿, 2:蓝) / Select color index to detect
        color_index = 0  # 可以修改这个值来选择检测不同的颜色
        threshold = THRESHOLDS[color_index]
        detect_color = get_closest_rgb(threshold)

        while True:
            clock.tick()
            img = sensor.snapshot()

            # 检测指定颜色 / Detect specified color
            blobs = img.find_blobs([threshold])
            if blobs:
                process_blobs(img, blobs, detect_color)

            fps = clock.fps()
            draw_fps(img, fps)
            print(fps)

            Display.show_image(img)

    except KeyboardInterrupt as e:
```

```
        print("用户中断 / User interrupted: ", e)
    except Exception as e:
        print(f"发生错误 / Error occurred: {e}")
    finally:
        if 'sensor' in locals() and isinstance(sensor, Sensor):
            sensor.stop()
        Display.deinit()
        MediaManager.deinit()

if __name__ == "__main__":
    main()
```

代码结构

程序设置：

- 设定显示屏分辨率为 800×480
- 定义了4种颜色的LAB色彩空间阈值（红、绿、蓝和一个自定义颜色）

主要功能：

- 初始化摄像头和显示屏
- 实时捕获视频画面
- 检测画面中特定颜色的物体
- 在检测到的物体周围画框并标记中心点
- 显示当前帧率(FPS)

工作流程：

- 程序启动后选择要检测的颜色（通过color_index设置）
- 持续捕获摄像头画面
- 使用LAB色彩空间进行颜色识别
- 在检测到的目标周围绘制矩形框和十字标记
- 实时显示处理后的图像和帧率

异常处理：

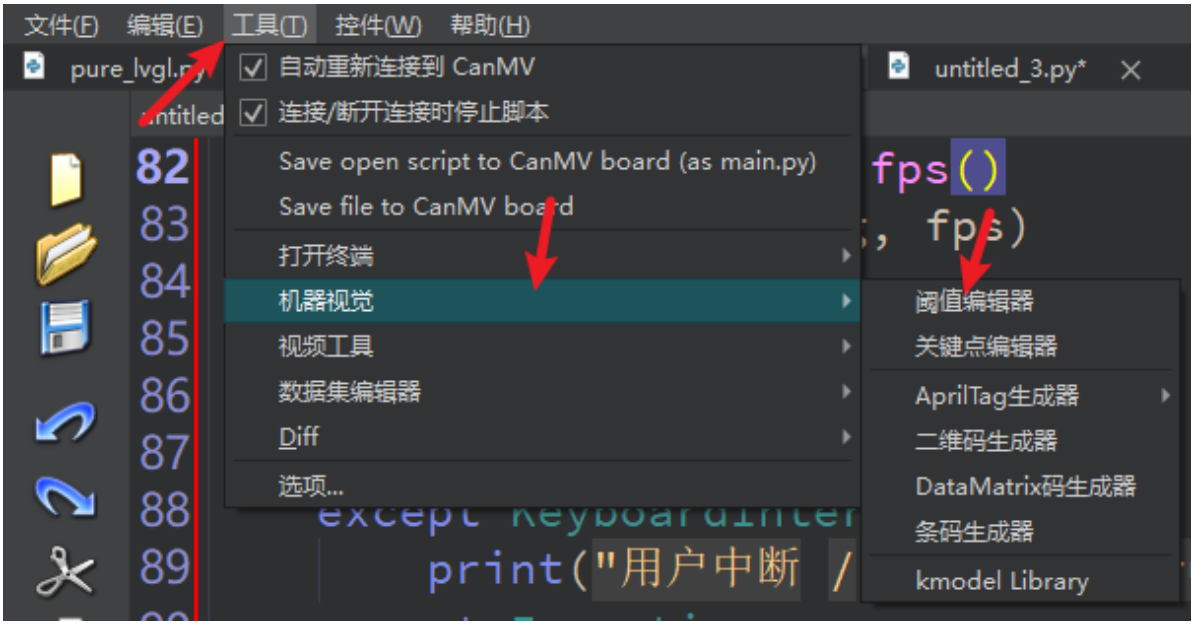
- 包含了用户中断和其他异常的处理机制
- 程序结束时会正确释放摄像头和显示设备资源

添加自定义颜色

CanMV IDE为我们提供了一个比较方便的LAB色域选择工具

我们点击CanMV IDE左上角的【工具】->【机器视觉】->【阈值编辑器】

打开一张【存在】我们想要识别的颜色的图片（干扰颜色尽可能多）

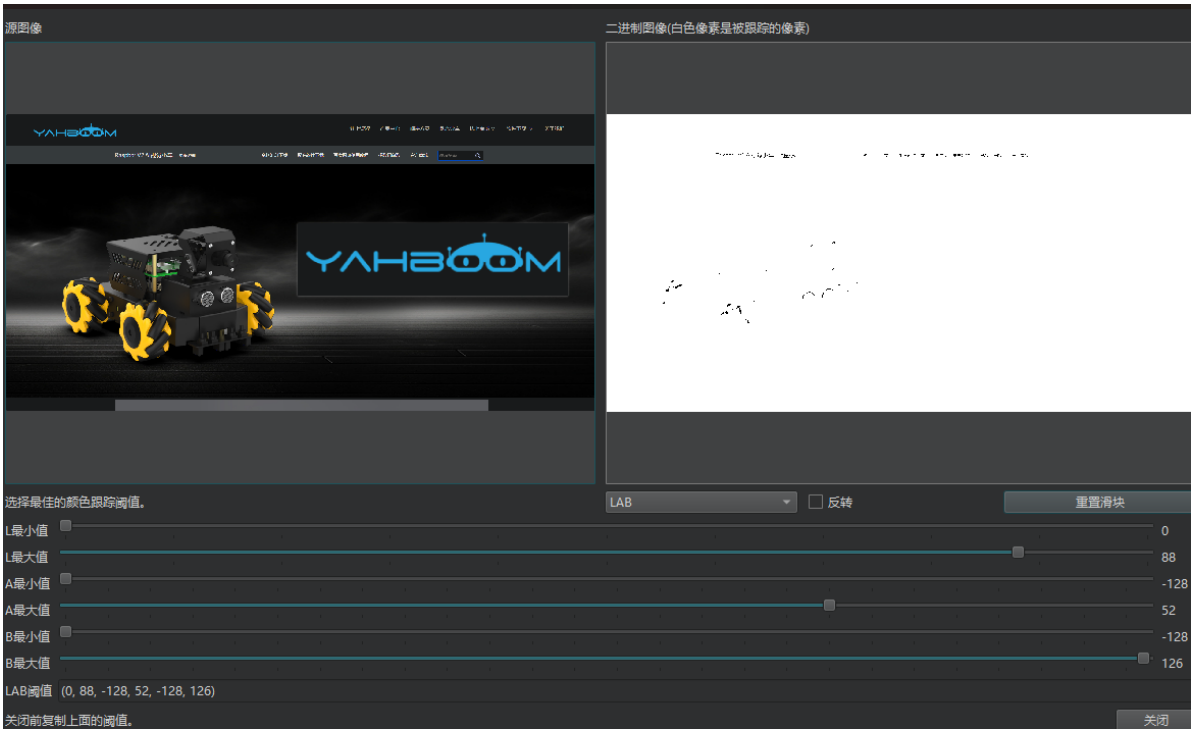


以亚博智能的Logo的颜色为例

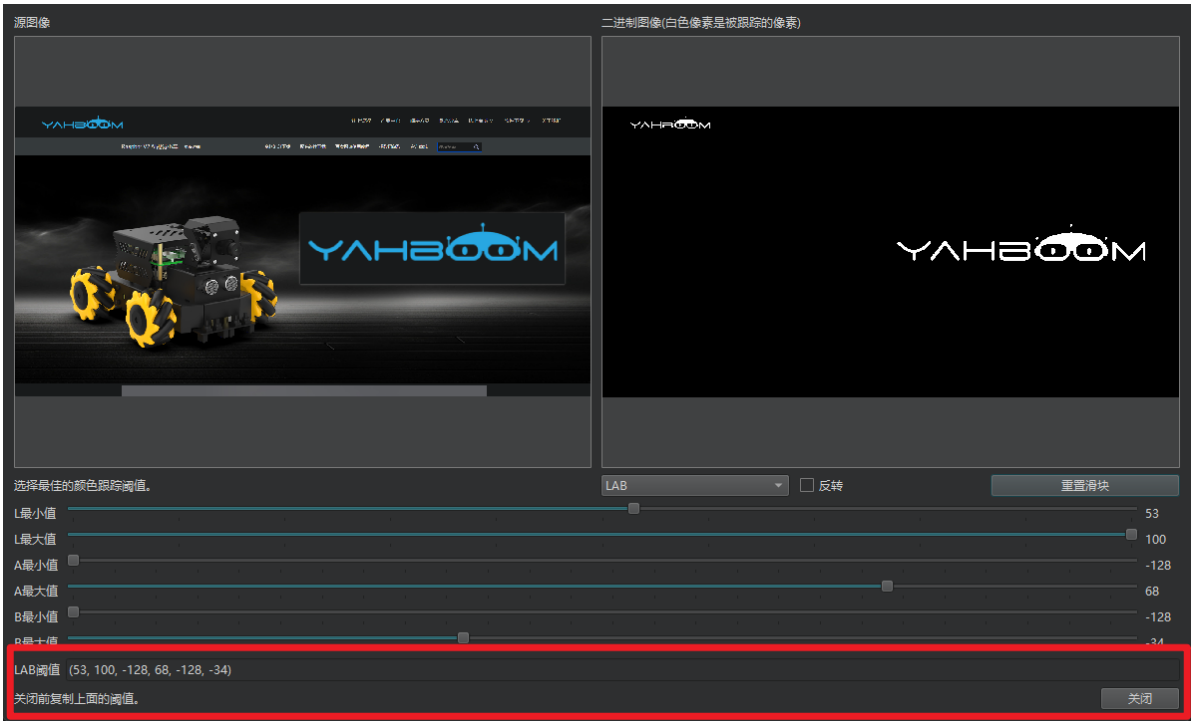
原始图像：



用阈值编辑器打开：



拖动下方滑块，调整到右边的二进制图像中，只有我们需要的颜色的部分是白色，其余地方是黑色



此时红圈中的LAB阈值就是我们需要的

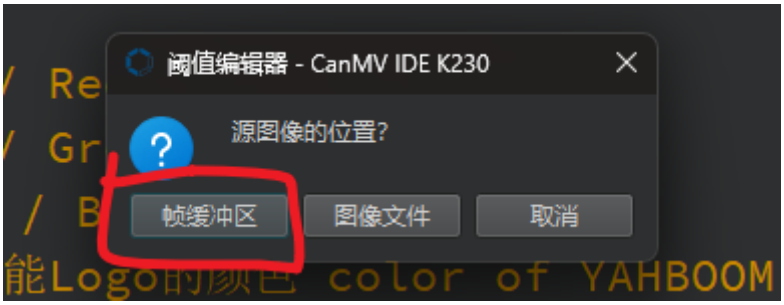
我们将这个阈值复制后添加到代码中的THRESHOLDS数组中

```
THRESHOLDS = [
    (30, 100, 15, 127, 15, 127),     # 红色阈值 / Red threshold
    (30, 100, -64, -8, 50, 70),      # 绿色阈值 / Green threshold
    (0, 40, 0, 90, -128, -20),        # 蓝色阈值 / Blue threshold
    (53, 100, -128, 68, -128, -34)     # 亚博智能Logo的颜色 color of YAHBOOM
]
```

修改color_index为我们刚刚添加的这个颜色在数组中的下标

```
# 选择要检测的颜色索引 (0:红, 1:绿, 2:蓝) / Select color index to detect
color_index = 3  # 可以修改这个值来选择检测不同的颜色
threshold = THRESHOLDS[color_index]
detect_color = get_closest_rgb(threshold)
```

这样我们的程序就可以开始检测并追踪我们新增的这个颜色了

> **颜色检测效果不佳?**
>
> 1. 尽量在打开阈值编辑器的时候，用帧缓冲区，或者使用K230拍照后的图像文件。这样能大大提升识别的准确度
>
> 
>
>
> 2. 可以放宽一点识别的阈值